

1 Optimal Parking with Stochastic Availability

A driver is looking for inexpensive parking on the way to his destination. The parking area contains N spaces, and a garage at the end. The driver starts at space 0 and traverses the parking spaces sequentially. Each parking space k costs $c(k)$ and is free with probability $p(k)$, independently of other spaces. If the driver reaches the last parking space $N - 1$ and does not park there, he must park at the garage, which costs C .

The driver can observe whether a parking space is free only when he reaches it. If the space is free, he may decide to park or continue to the next space. The objective is to find a parking policy that maximizes the expected reward (or equivalently, minimizes the expected cost).

(a)

Formulate this problem as a dynamic programming problem with N stages (fixed-horizon DP). Specify the set of all possible states S_k at stage k , and the set of all possible actions $A_k(s_k)$ available at stage k for a given state.

Stage S_k , where $k = 0, 1, \dots, N - 1$, represents the moment when the driver arrives at parking space k and observes whether it is free.

$$S_k = \{(k, Free), (k, Occupied)\}$$

And if the driver arrive at stage N (garage),he must stop here.

So $S_k = \{(k, Parked)\}$ for $k=N$.

Action A_k , represents whether the driver decide to park his car at certain stage or not.

$$A_k(S_k(k, Free)) = \{Go, Park\} \text{ and } A_k(S_k(k, Occupied)) = \{Go\}$$

If at stage N , or it is already parked, he can not move.

$$\text{So } A_k(S_k(k, Parked)) = \{NoGo\}$$

Description: If the driver chooses Park at space k . If he chooses Go, it moves to stage $k + 1$. The next space is free with probability $p(k + 1)$ and occupied with probability $1 - p(k + 1)$. If the driver reaches stage $N - 1$ and does not park, he must park at the garage and pay cost C .

(b)

Specify the state-reward transition distribution

$$p_k(s', r \mid s, a) = \mathbb{P}(S_{k+1} = s', R_{k+1} = r \mid S_k = s, A_k = a).$$

You may define the reward as negative cost and maximize expected return, or define the reward as cost and minimize expected return.

Case 1: already parked

For $s=\text{Parked}$, $a=\text{NoGo}$

$$p_k(s', r \mid s, a) = p_k((k+1, \text{Parked}), 0 \mid (k, \text{Parked}), \text{NoGo}) = 1$$

Case 2: space is free, choose park

For $s=\text{Free}$, $a=\text{park}$,

$$p_k(s', r \mid s, a) = p_k((k+1, \text{Parked}), -c(k) \mid (k, \text{Free}), \text{Park}) = 1$$

Case 3: space is free, choose go to next

For $s=\text{Free}$, $a=\text{Go}$

If $k \leq N-2$

$$p_k((k+1, \text{Free}), 0 \mid (k, \text{Free}), \text{Go}) = p(k+1),$$

$$p_k((k+1, \text{Occupied}), 0 \mid (k, \text{Free}), \text{Go}) = 1 - p(k+1),$$

If $k = N-1$

$$p_k((k+1, \text{Parked}), -C \mid (k, \text{Free}), \text{Go}) = 1$$

Case 4: space is occupied, have to go to next

For $s=\text{Occupied}$, $a=\text{Go}$

If $k \leq N-2$

$$p_k((k+1, \text{Free}), 0 \mid (k, \text{Occupied}), \text{Go}) = p(k+1),$$

$$p_k((k+1, \text{Occupied}), 0 \mid (k, \text{Occupied}), \text{Go}) = 1 - p(k+1),$$

If $k = N-1$

$$p_k((k+1, \text{Parked}), -C \mid (k, \text{Occupied}), \text{Go}) = 1$$

(c)

Write down the dynamic programming algorithm explicitly in terms of N , C , $c(k)$, and $p(k)$.

Define: J_k^s : optimal expected cost starting at space k given stage s .

We start from the last stage: when $k = N-1$

If the last space is occupied, the driver must park in the garage:

$$J_k^O = C.$$

If the last space is free, the driver chooses the cheaper option:

$$J_k^F = \min\{c(k), C\}.$$

Recursion: when $k = N-2, \dots, 0$

If space k is occupied, the driver must go to $k+1$, hence

$$J_k^O = p(k+1) J_{k+1}^F + (1 - p(k+1)) J_{k+1}^O.$$

If space k is free, the driver can park or go, hence

$$J_k^F = \min\left\{c(k), p(k+1) J_{k+1}^F + (1 - p(k+1)) J_{k+1}^O\right\}.$$

Initial: when $k=0$

$$J_0 = p(0) J_0^F + (1 - p(0)) J_0^O.$$

(d)

Suppose that $c(t)$ is monotonically decreasing as t increases. Show that the optimal policy is a threshold policy: the driver should skip parking until reaching a certain space and then park at the first available space thereafter.

(Hint: Define the value of entering space k as

$$V_k^* := p(k)v_k^*(\text{space } k \text{ is free}) + (1 - p(k))v_k^*(\text{space } k \text{ is not free}),$$

where $v_k^*(\cdot)$ is the optimal value function at stage k . Depending on whether the problem is formulated as a maximization or minimization problem, show that V_k^* is monotonic in k .)

$$\text{Define } V_k^* = p(k)v_k^*(F) + (1 - p(k))v_k^*(O)$$

According to (c), we can get:

$$\begin{aligned} \text{if space } k \text{ is free, } v_k^* &= \min\{c(k), p(k+1)v_{k+1}^*(F) + (1 - p(k+1))v_{k+1}^*(O)\} \\ &= \min\{c(k), V_{k+1}^*\} \end{aligned}$$

$$\begin{aligned} \text{if space } k \text{ is not free, } v_k^* &= p(k+1)v_{k+1}^*(F) + (1 - p(k+1))v_{k+1}^*(O) \\ &= V_{k+1}^* \end{aligned}$$

$$\text{So we can rewrite } V_k^* = p(k)\min\{c(k), V_{k+1}^*\} + (1 - p(k))V_{k+1}^*$$

Because $c(k)$ decreases as k increases, it will reach a certain point $k = k^*$ let $c(k) \leq V_{k+1}^*$, it means park at k^* will be the optimal choice.

Therefore, the optimal policy is:

for $k < k^*$: skip even if space k is free;

for $k \geq k^*$: park at the first free space encountered.

Thus the optimal policy is a threshold policy.

2 Machine Maintenance, Repair, and Replacement

Suppose we have a machine that is either running or broken down. If it runs through one week, it generates a gross profit of \$120; if it fails during the week, the gross profit is zero.

If the machine is running at the start of the week and preventive maintenance is performed, the probability of failure during the week is 0.4. If maintenance is not performed, the probability of failure is 0.7. Preventive maintenance costs \$20.

If the machine is broken at the start of the week, it may either be repaired at a cost of \$60, after which it fails during the week with probability 0.4, or it may be replaced at a cost of \$120 by a new machine that is guaranteed to run through its first week.

(a)

Find the optimal repair, replacement, and maintenance policy that maximizes total profit over three weeks, assuming that:

- At the start of the first week, the machine is running.
- The failure probability is 0.4 if maintenance is performed and 0.7 otherwise.
- There is no terminal penalty for the machine's condition at the end of the horizon.

Stage S_t : at the start of the week $t=1,2,3$, the stage of machine is either running or broken.

$$S_t = \{(t, \text{Running}), (t, \text{Broken})\}$$

Action A_t : at the start of the week, if the machine is running, we can choose to do a maintenance on it or not; and if the machine is broken, we can choose to do a repair on it or choose to replace it.

$$A_t(\text{Running}) = \{\text{Maintenance}, \text{Not Maintenance}\},$$

$$A_t(\text{Broken}) = \{\text{Repair}, \text{Replace}\}$$

Then write down the transition distribution:

$$p_t(s', r \mid s, a) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

Week 1: $S_1 = \text{Running}$, we can get the following situation:

$$p_1(\text{Running}, 100 \mid \text{Running}, \text{Maintenance}) = 0.6$$

$$p_1(\text{Broken}, -20 \mid \text{Running}, \text{Maintenance}) = 0.4$$

$$p_1(\text{Running}, 120 \mid \text{Running}, \text{Not Maintenance}) = 0.3$$

$$p_1(\text{Broken}, 0 \mid \text{Running}, \text{Not Maintenance}) = 0.7$$

Week 2 and Week 3 are the same, for $t=2,3$, we can get the following situation:

$$p_t(\text{Running}, 100 \mid \text{Running}, \text{Maintenance}) = 0.6$$

$$p_t(\text{Broken}, -20 \mid \text{Running}, \text{Maintenance}) = 0.4$$

$$p_t(\text{Running}, 120 \mid \text{Running}, \text{Not Maintenance}) = 0.3$$

$$p_t(\text{Broken}, 0 \mid \text{Running}, \text{Not Maintenance}) = 0.7$$

$$\begin{aligned}
p_t(Running, 60 \mid Broken, Repair) &= 0.6 \\
p_t(Broken, -60 \mid Broken, Repair) &= 0.4 \\
p_t(Running, 0 \mid Broken, Replace) &= 1
\end{aligned}$$

We can calculate V_1^* by backward iteration, since no penalty, $V_4(R) = V_4(B) = 0$
For week 3, $t=3$.

$$\begin{aligned}
V_3^M(R) &= 0.6 * 100 - 0.4 * 20 = 52, \quad V_3^{NM}(R) = 120 * 0.3 + 0 = 40 \\
V_3^{Repair}(B) &= 0.6 * 60 - 0.4 * 60 = 12, \quad V_3^{Replace}(B) = 0 \\
\text{So } V_3^*(R) &= 52 \quad (A = Maintenance), \quad V_3^*(B) = 12 \quad (A = Repair)
\end{aligned}$$

For week 2, $t=2$.

$$\begin{aligned}
V_2^M(R) &= 0.6 * (100 + 52) + 0.4 * (-20 + 12) = 88, \quad V_2^{NM}(R) = (120 + 52) * 0.3 + (0 + 12) * 0.7 = 60 \\
V_2^{Repair}(B) &= 0.6 * (60 + 52) + 0.4 * (-60 + 12) = 48, \quad V_2^{Replace}(B) = 52 * 1 = 52 \\
\text{So } V_2^*(R) &= 88 \quad (A = Maintenance), \quad V_2^*(B) = 52 \quad (A = Replace)
\end{aligned}$$

For week 1, $t=1$.

$$\begin{aligned}
V_1^M(R) &= 0.6 * (100 + 88) + 0.4 * (-20 + 52) = 125.6, \quad V_1^{NM}(R) = (120 + 88) * 0.3 + (0 + 52) * 0.7 = 98.8 \\
\text{So } V_1^*(R) &= 125.6 \quad (A = Maintenance)
\end{aligned}$$

Therefore, the optimal policy is always choose maintenance if the machine is running at the start of week. But if machine is broken, choose replace in week 2 and repair in week 3.

(b)

Now assume that there is a penalty of \$100 if the machine is broken at the end of the third week. Re-compute the optimal policy for the last stage. Does the optimal policy change compared to part (a)?

$$V_4(R) = 0, \text{ But } V_4(B) = -100$$

In this case, For week 3, $t=3$.

$$\begin{aligned}
V_3^M(R) &= 0.6 * 100 + 0.4 * (-20 - 100) = 12, \quad V_3^{NM}(R) = 120 * 0.3 + (-100) * 0.7 = -34 \\
V_3^{Repair}(B) &= 0.6 * 60 + 0.4 * (-60 - 100) = -28, \quad V_3^{Replace}(B) = 0 \\
\text{So } V_3^*(R) &= 12 \quad (A = Maintenance), \quad V_3^*(B) = 0 \quad (A = Replace)
\end{aligned}$$

Therefore, if the machine is running at the start of week 3, the optimal policy does not change; it is still maintenance. But if it is broken, the optimal policy changes from repair to replace.

3 Finite-Horizon Deterministic Control Problem

Consider the system

$$s_{t+1} = s_t + a_t, \quad t = 0, 1, 2, 3,$$

with initial state $s_0 = 1$, and return function

$$\sum_{t=0}^3 (s_t^2 - 2a_t^2).$$

The action constraint is

$$a_t \in \{a : 0 \leq s_t + a \leq 5, a \text{ is integer}\}.$$

Apply the dynamic programming algorithm to find the optimal action sequence

$$a_0^*, a_1^*, a_2^*, a_3^*$$

that maximizes the return function. (You may solve this problem using a computer program if you wish. If so, write down the DP equations analytically and include the code with your solution.)

we can calculate π^* by backward recursion:

The terminal point $V_4(s) = 0$

$$V_k^*(s) = \max_{a_t \in a_t^*} \sum_{t=k}^3 (s_t^2 - 2a_t^2)$$

So the Bellman optimal equation:

$$V_t^*(s) = \max_{a_t \in a_t^*} [s_t^2 - 2a_t^2 + V_{t+1}^*(s_{t+1})], (s_{t+1} = s_t + a_t)$$

$$a_t^*(s) = \operatorname{argmax}_{a_t \in a_t^*} [s_t^2 - 2a_t^2 + V_{t+1}^*(s_{t+1})]$$

Limit: $0 \leq s_t + a_t \leq 5$

Backward recursion: when $t=3$

$$\because V_4(s) = 0, a_3^* = \operatorname{argmax} [s_3^2 - 2a_3^2]$$

To make it max, obviously $a_3^* = 0$, Then $V_3^*(s) = s_3^2$

when $t=2$

$$a_2^* = \operatorname{argmax} [s_2^2 - 2a_2^2 + V_3^*(s)] = \operatorname{argmax} [s_2^2 - 2a_2^2 + (s_2 + a_2)^2]$$

$$= \operatorname{argmax} [-a_2^2 + 2s_2a_2 + 2s_2^2] = \operatorname{argmax} [-(a_2 - s_2)^2 + 3s_2^2]$$

so we can get when $a_2 = s_2$, it gets maximum. Then $V_2^*(s) = 3s_2^2$

$$\text{when } t=1, a_1^* = \operatorname{argmax} [s_1^2 - 2a_1^2 + V_2^*(s)] = \operatorname{argmax} [s_1^2 - 2a_1^2 + 3(s_1 + a_1)^2]$$

$$= \operatorname{argmax} [a_1^2 + 6s_1a_1 + 4s_1^2]$$

so we can get when $a_1 = -3s_1$, it gets maximum. Then $V_1^*(s) = -5s_1^2$

$$\text{when } t=0, a_0^* = \operatorname{argmax} [s_0^2 - 2a_0^2 + V_1^*(s)] = \operatorname{argmax} [s_0^2 - 2a_0^2 - 5(s_0 + a_0)^2]$$

$$= \operatorname{argmax} [-7a_0^2 - 10s_0a_0 - 4s_0^2]$$

So we can get $a_0 = -5/7s_0$

```

1 states = range(0, 6)      # s ∈ {0,1,2,3,4,5}
2 T = 4                    # t = 0,1,2,3
3
4 def actions(s):
5     # a is integer and must keep next state in [0,5]
6     return [a for a in range(-5, 6) if 0 ≤ s + a ≤ 5]
7
8 def reward(s, a):
9     return s*s - 2*a*a
10
11 # V[t][s] = value from time t at state s
12 V = [{s: 0 for s in states} for _ in range(T + 1)]
13 pi = [{s: None for s in states} for _ in range(T)]
14
15 # backward DP
16 for t in reversed(range(T)):
17     for s in states:
18         best_val = -10**18
19         best_a = None
20         for a in actions(s):
21             sp = s + a
22             val = reward(s, a) + V[t + 1][sp]
23             if val > best_val:
24                 best_val = val
25                 best_a = a
26         V[t][s] = best_val
27         pi[t][s] = best_a
28 # rollout from s0=1
29 s = 1
30 aseq = []
31 ret = 0
32 for t in range(T):
33     a = pi[t][s]
34     aseq.append(a)
35     ret += reward(s, a)
36     s = s + a
37
38 print("optimal actions:", aseq)
39 print("optimal return:", ret)
40

```

We can solve it by a program:

The result: optimal action $[a_0^*, a_1^*, a_2^*, a_3^*] = [3, 1, 0, 0]$
 optimal return=47